

Entity Embedding

ACTL3143 & ACTL5111 Deep Learning for Actuaries
Patrick Laub

Lecture Outline

- **Entity Embedding**
- Categorical Variables & Entity Embeddings
- Keras' Functional API
- French Motor Dataset with Embeddings
- Scale by Exposure

Revisit the French motor dataset

	IDpol	ClaimNb	Exposure	Area	VehPower	VehAge	DrivAge	Bon
0	1.0	1.0	0.10000	D	5.0	0.0	55.0	50.0
1	3.0	1.0	0.77000	D	5.0	0.0	55.0	50.0
...	...	...	...	...	...	...	...	...
678011	6114329.0	0.0	0.00274	B	4.0	0.0	60.0	50.0
678012	6114330.0	0.0	0.00274	B	7.0	6.0	29.0	54.0

678013 rows × 12 columns

Data dictionary

Variable	Description	Preprocessing
IDpol	Policy number (unique identifier)	Dropped
ClaimNb	Number of claims on the given policy	Target
Exposure*	Total exposure in yearly units	Normalised
Area	Area code (ordinal)	Ordinal Encode
VehPower	Power of the car (ordinal encoded)	Normalised
VehAge	Age of the car in years	Normalised
DrivAge	Age of the (most common) driver in years	Normalised
BonusMalus	Bonus–malus level between 50 and 230 (with reference level 100)	Normalised
VehBrand*	Car brand (nominal)	One-hot
VehGas	Diesel or regular fuel car (binary)	One-hot
Density	Density of inhabitants per km ² in the city of the living place of the driver	Normalised
Region*	Regions in France (prior to 2016)	One-hot

Source: Nell et al. (2020), [Case Study: French Motor Third-Party Liability Claims](#), SSRN.

The model

Have $\{(\mathbf{x}_i, y_i)\}_{i=1, \dots, n}$ for $\mathbf{x}_i \in \mathbb{R}^{47}$ and $y_i \in \mathbb{N}_0$.

Assume the distribution

$$Y_i \sim \text{Poisson}(\lambda(\mathbf{x}_i))$$

We have $\mathbb{E}Y_i = \lambda(\mathbf{x}_i)$. The NN takes $\mathbf{x}_i$ & predicts $\mathbb{E}Y_i$.

i Note

For insurance, *this is a bit weird*. The exposures are different for each policy.

$\lambda(\mathbf{x}_i)$ is the expected number of claims for the duration of policy i 's contract.

Normally, $\text{Exposure}_i \notin \mathbf{x}_i$, and $\lambda(\mathbf{x}_i)$ is the expected rate *per year*, then

$$Y_i \sim \text{Poisson}(\text{Exposure}_i \times \lambda(\mathbf{x}_i)).$$

What values do we see in the data?

```
1 X_train_raw["Area"].value_counts()
2 X_train_raw["VehBrand"].value_counts()
3 X_train_raw["VehGas"].value_counts()
4 X_train_raw["Region"].value_counts()
```

Area

```
C    5514
D    4116
```

...

```
B    2387
F     444
```

Name: count, Length: 6, dtype: int64

VehGas

```
Regular    10658
Diesel      8092
```

Name: count, dtype: int64

VehBrand

```
B1    4998
B2    4906
```

...

```
B11    283
B14    140
```

Name: count, Length: 11, dtype: int64

Region

```
R24    6493
R82    2112
```

...

```
R42     48
R43     26
```

Name: count, Length: 22, dtype: int64

How we preprocessed last time

```

1 from sklearn.compose import make_column_transformer
2
3 ct = make_column_transformer(
4     (OneHotEncoder(sparse_output=False, drop="first"), ["VehGas", "VehBrand", "Region"]),
5     (OrdinalEncoder(), ["Area"]),
6     remainder=StandardScaler(),
7     verbose_feature_names_out=False
8 )
9 X_train = ct.fit_transform(X_train_raw)

```

```
1 X_train_raw.head(3)
```

	Exposure	Area	VehPower	VehAge
0	1.00	A	7.0	8.0
1	0.79	B	7.0	7.0
2	1.00	C	6.0	13.0

```
1 X_train.head(3)
```

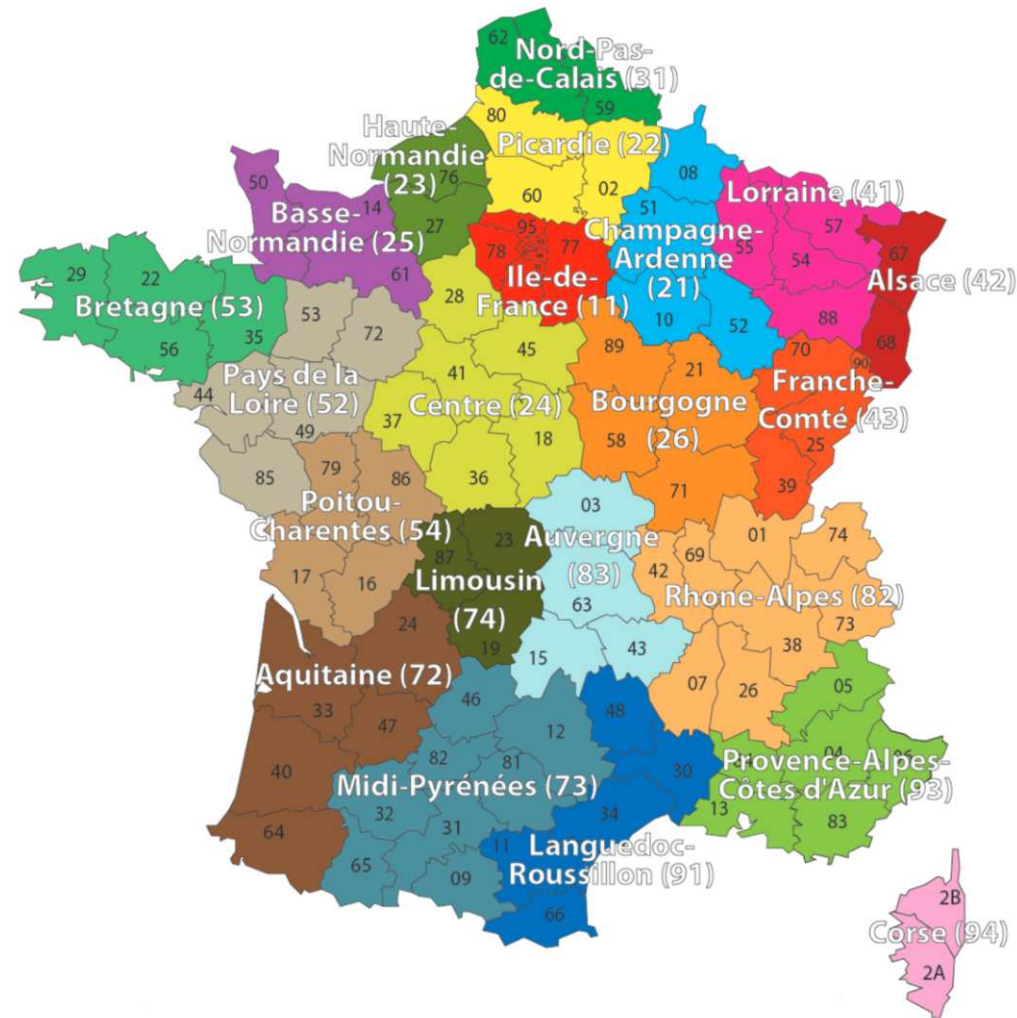
	VehGas_Regular	VehBrand_B10	Ve
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	1.0	0.0	0.0

3 rows × 39 columns

Lecture Outline

- Entity Embedding
- **Categorical Variables & Entity Embeddings**
- Keras' Functional API
- French Motor Dataset with Embeddings
- Scale by Exposure

Region column



French Administrative Regions

Source: [Wikimedia](#)

One-hot encoding

```

1 oh = OneHotEncoder(sparse_output=False)
2 X_train_oh = oh.fit_transform(X_train_raw[["Region"]])
3 X_test_oh = oh.transform(X_test_raw[["Region"]])
4 print(list(X_train_raw["Region"][:5]))
5 X_train_oh.head()

```

['R24', 'R21', 'R53', 'R24', 'R82']

	Region_R11	Region_R21	Region_R22	Region_R23	Region_R24	Region_R25
0	0.0	0.0	0.0	0.0	1.0	0.0
1	0.0	1.0	0.0	0.0	0.0	0.0
...	...	...	...	...	...	...
3	0.0	0.0	0.0	0.0	1.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 22 columns

Train on one-hot inputs

```
1 num_regions = len(oh.categories_[0])
2
3 random.seed(12)
4 model = Sequential([
5     Dense(2, input_dim=num_regions),
6     Dense(1, activation="exponential")
7 ])
8
9 model.compile(optimizer="adam", loss="poisson")
10
11 es = EarlyStopping(verbose=True)
12 hist = model.fit(X_train_oh, y_train, epochs=100, verbose=0,
13                 validation_split=0.2, callbacks=[es])
14 hist.history["val_loss"][-1]
```

Epoch 6: early stopping

0.7679625153541565

Make a fake batch of data

```
1 X = np.eye(num_regions)
2 pd.DataFrame(X, columns=oh.categories_[0])
```

	R11	R21	R22	R23	R24	R25	R26
0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	1.0	0.0	0.0	0.0	0.0	0.0
...	...	...	...	...	...	...	...
20	0.0	0.0	0.0	0.0	0.0	0.0	0.0
21	0.0	0.0	0.0	0.0	0.0	0.0	0.0

22 rows × 22 columns

```
1 model.layers[0](X)
```

```
tensor([[ -0.0646, -0.2346],
        [ 0.1463, -0.0323],
        [-0.3050, -0.2913],
        [ 0.0257, -0.3189],
        [ 0.3848, -0.2778],
        [ 0.0983, -0.4137],
        [ 0.2235, -0.2502],
        [ 0.1192,  0.3202],
        [ 0.6295, -0.4385],
        [ 0.2008, -0.0306],
        [-0.2030, -0.3797],
        [ 0.7612, -0.2256],
        [ 0.5068, -0.0436],
        [ 0.7867, -0.4326],
        [-0.2026, -0.1788],
        [-0.2430, -0.0941],
```

The first layer

```
1 layer = model.layers[0]
2 W, b = layer.get_weights()
3 X.shape, W.shape, b.shape
```

```
((22, 22), (22, 2), (2,))
```

```
1 X @ W + b
```

```
array([[ -0.06, -0.23],
       [ 0.15, -0.03],
       [-0.3 , -0.29],
       [ 0.03, -0.32],
       [ 0.38, -0.28],
       [ 0.1 , -0.41],
       [ 0.22, -0.25],
       [ 0.12,  0.32],
       [ 0.63, -0.44],
       [ 0.2 , -0.03],
       [-0.2 , -0.38],
       [ 0.76, -0.23],
       [ 0.51, -0.04],
       [ 0.79, -0.43],
       [-0.2 , -0.18],
       [-0.24, -0.09],
```

```
1 W + b
```

```
array([[ -0.06, -0.23],
       [ 0.15, -0.03],
       [-0.3 , -0.29],
       [ 0.03, -0.32],
       [ 0.38, -0.28],
       [ 0.1 , -0.41],
       [ 0.22, -0.25],
       [ 0.12,  0.32],
       [ 0.63, -0.44],
       [ 0.2 , -0.03],
       [-0.2 , -0.38],
       [ 0.76, -0.23],
       [ 0.51, -0.04],
       [ 0.79, -0.43],
       [-0.2 , -0.18],
       [-0.24, -0.09],
```

Just a look-up operation

```
1 display(list(oh.categories_[0]))
```

```
['R11',  
'R21',  
'R22',  
'R23',  
'R24',  
'R25',  
'R26',  
'R31',  
'R41',  
'R42',  
'R43',  
'R52',  
'R53',  
'R54',  
'R72',  
'R73',
```

```
1 W + b
```

```
array([[ -0.06, -0.23],  
       [ 0.15, -0.03],  
       [-0.3 , -0.29],  
       [ 0.03, -0.32],  
       [ 0.38, -0.28],  
       [ 0.1 , -0.41],  
       [ 0.22, -0.25],  
       [ 0.12,  0.32],  
       [ 0.63, -0.44],  
       [ 0.2 , -0.03],  
       [-0.2 , -0.38],  
       [ 0.76, -0.23],  
       [ 0.51, -0.04],  
       [ 0.79, -0.43],  
       [-0.2 , -0.18],  
       [-0.24, -0.09],
```

Turn the region into an index

```
1 oe = OrdinalEncoder()  
2 X_train_reg = oe.fit_transform(X_train_raw[["Region"]])  
3 X_test_reg = oe.transform(X_test_raw[["Region"]])  
4  
5 for i, reg in enumerate(oe.categories_[0][:3]):  
6     print(f"The Region value {reg} gets turned into {i}.")
```

The Region value R11 gets turned into 0.

The Region value R21 gets turned into 1.

The Region value R22 gets turned into 2.

Use an Embedding layer

```

1 from keras.layers import Embedding
2 num_regions = X_train_raw["Region"].nunique()
3
4 random.seed(12)
5 model = Sequential([
6     Embedding(input_dim=num_regions, output_dim=2),
7     Dense(1, activation="exponential")
8 ])
9
10 model.compile(optimizer="adam", loss="poisson")

```

```

1 es = EarlyStopping(verbose=True)
2 hist = model.fit(X_train_reg, y_train, epochs=100, verbose=0,
3     validation_split=0.2, callbacks=[es])
4 hist.history["val_loss"][-1]

```

Epoch 4: early stopping

0.7677991390228271

```
1 model.layers
```

```
[<Embedding name=embedding, built=True>, <Dense name=dense_2, built=True>]
```


Keras' Embedding Layer

```
1 model.layers[0].get_weights()[0]
```

```
array([[ 0.05, -0.07],
       [-0.02,  0.03],
       [-0.04, -0.02],
       [ 0.09, -0.12],
       [ 0.24, -0.22],
       [ 0.16, -0.19],
       [ 0.17, -0.17],
       [-0.05,  0.09],
       [ 0.36, -0.33],
       [ 0.03, -0.01],
       [ 0.02, -0.07],
       [ 0.36, -0.3 ],
       [ 0.21, -0.16],
       [ 0.42, -0.37],
       [-0.02, -0.02],
       [-0.06,  0.02],
```

```
1 X_train_raw["Region"].head(4)
```

```
0    R24
1    R21
2    R53
3    R24
Name: Region, dtype: str
```

```
1 X_sample = X_train_reg[:4].to_numpy()
2 X_sample
```

```
array([[ 4.],
       [ 1.],
       [12.],
       [ 4.]])
```

```
1 enc_tensor = model.layers[0](X_sample)
2 keras.ops.convert_to_numpy(enc_tensor).sc
```

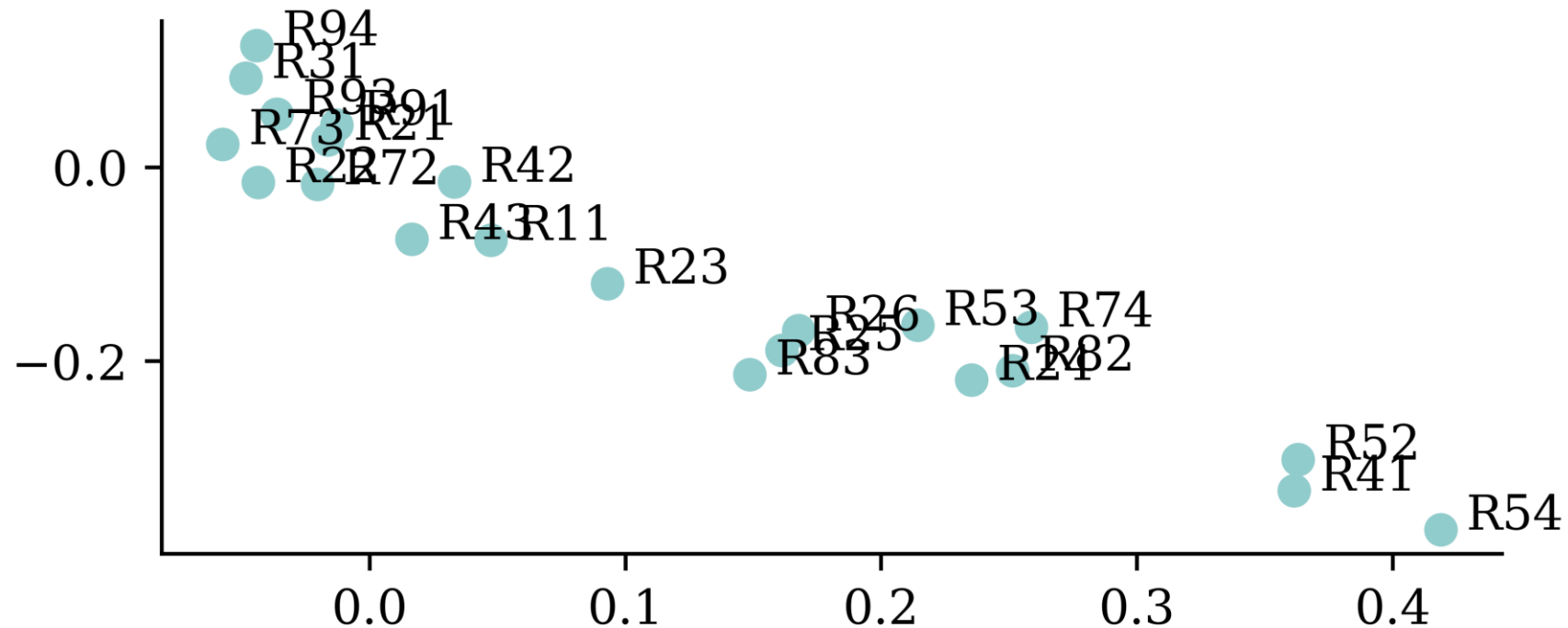
```
array([[ 0.24, -0.22],
       [-0.02,  0.03],
       [ 0.21, -0.16],
       [ 0.24, -0.22]], dtype=float32)
```

The learned embeddings

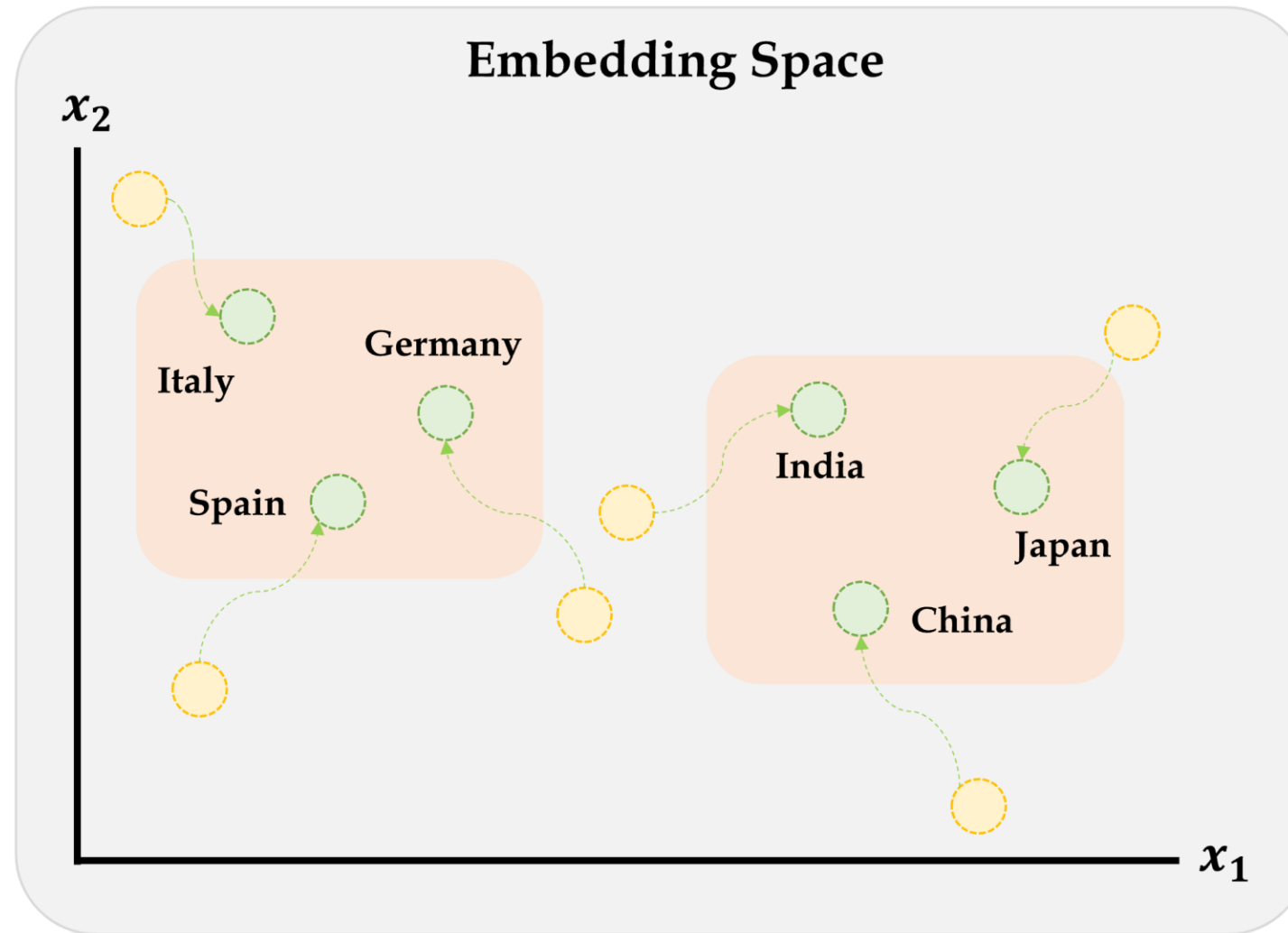
```

1 points = model.layers[0].get_weights()[0]
2 plt.scatter(points[:,0], points[:,1])
3 for i in range(num_regions):
4     plt.text(points[i,0]+0.01, points[i,1] , s=oe.categories_[0][i])

```



Entity embeddings



Embeddings will gradually improve during training.

Source: Marcus Lautier (2022).

Embeddings & other inputs

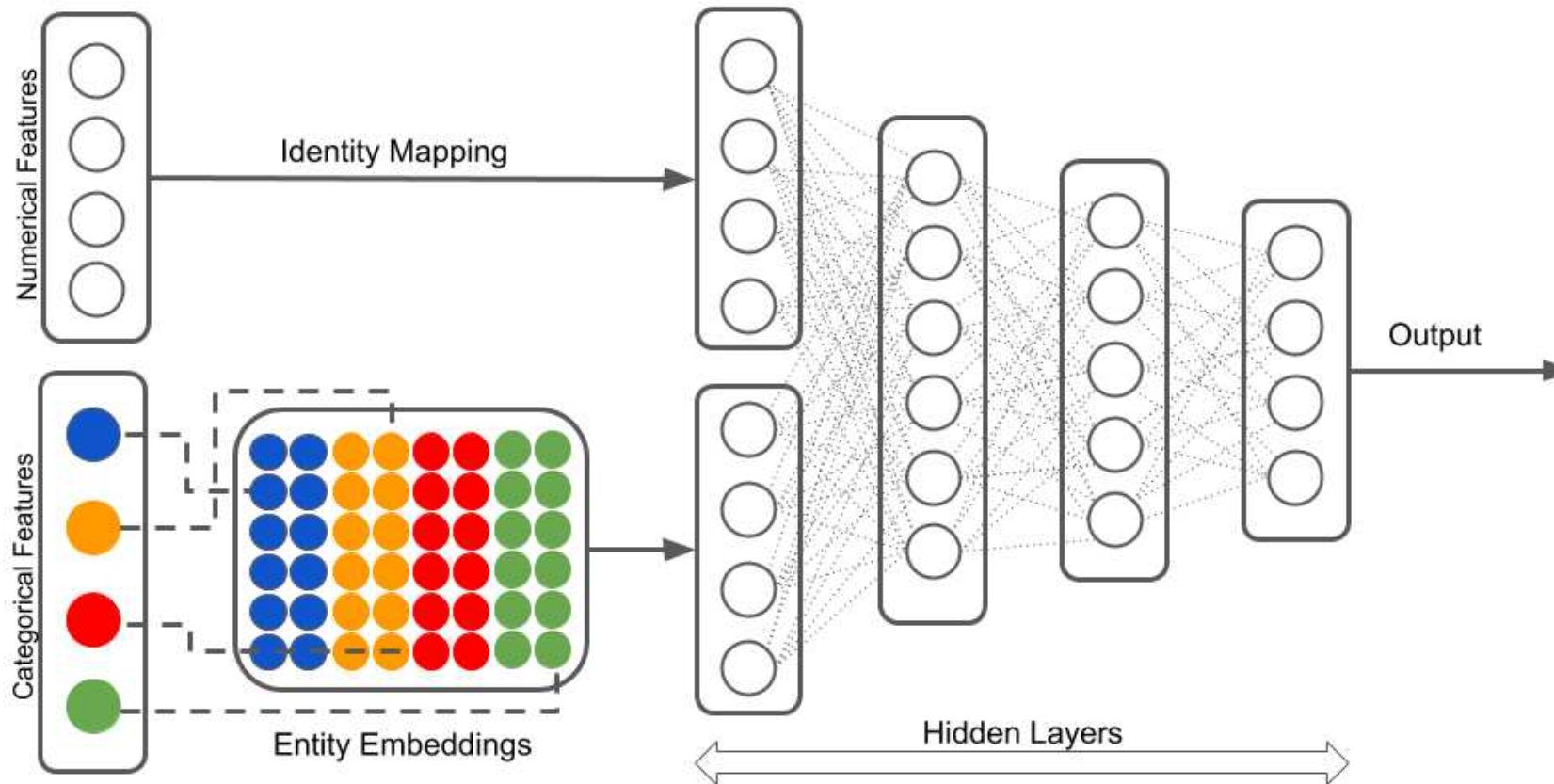


Illustration of a neural network with both continuous and categorical inputs.

We can't do this with Sequential models...

Source: LotusLabs Blog, [Accurate insurance claims prediction with Deep Learning](#).

Lecture Outline

- Entity Embedding
- Categorical Variables & Entity Embeddings
- **Keras' Functional API**
- French Motor Dataset with Embeddings
- Scale by Exposure

Converting Sequential models

```
1 from keras.models import Model
2 from keras.layers import Input
```

```
1 random.seed(12)
2
3 model = Sequential([
4     Dense(30, "leaky_relu"),
5     Dense(1, "exponential")
6 ])
7
8 model.compile(
9     optimizer="adam",
10    loss="poisson")
11
12 hist = model.fit(
13     X_train_oh, y_train,
14     epochs=1, verbose=0,
15     validation_split=0.2)
16 hist.history["val_loss"][-1]
```

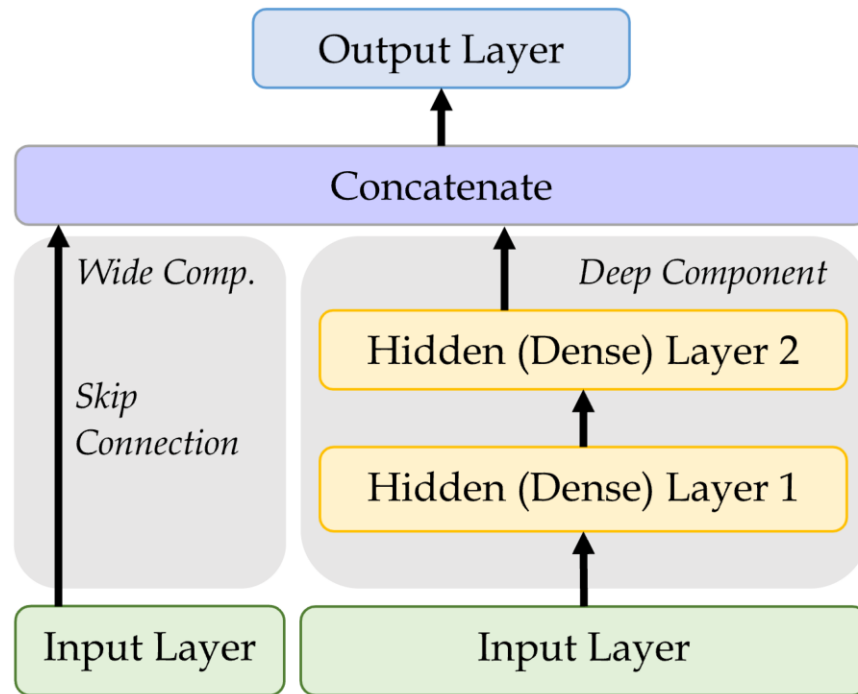
0.7695862650871277

```
1 random.seed(12)
2
3 inputs = Input(shape=(X_train_oh.shape[1]
4 x = Dense(30, "leaky_relu")(inputs)
5 out = Dense(1, "exponential")(x)
6 model = Model(inputs, out)
7
8 model.compile(
9     optimizer="adam",
10    loss="poisson")
11
12 hist = model.fit(
13     X_train_oh, y_train,
14     epochs=1, verbose=0,
15     validation_split=0.2)
16 hist.history["val_loss"][-1]
```

0.7695212364196777

See **one-length tuples**.

Wide & Deep network



Add a *skip connection* from input to output layers.

```

1 from keras.layers \
2     import Concatenate
3
4 inp = Input(shape=X_train.shape[1:])
5 hidden1 = Dense(30, "leaky_relu")(inp)
6 hidden2 = Dense(30, "leaky_relu")(hidden1)
7 concat = Concatenate()(
8     [inp, hidden2])
9 output = Dense(1)(concat)
10 model = Model(
11     inputs=[inp],
12     outputs=[output])

```

An illustration of the wide & deep network architecture.

Naming the layers

For complex networks, it is often useful to give meaningful names to the layers.

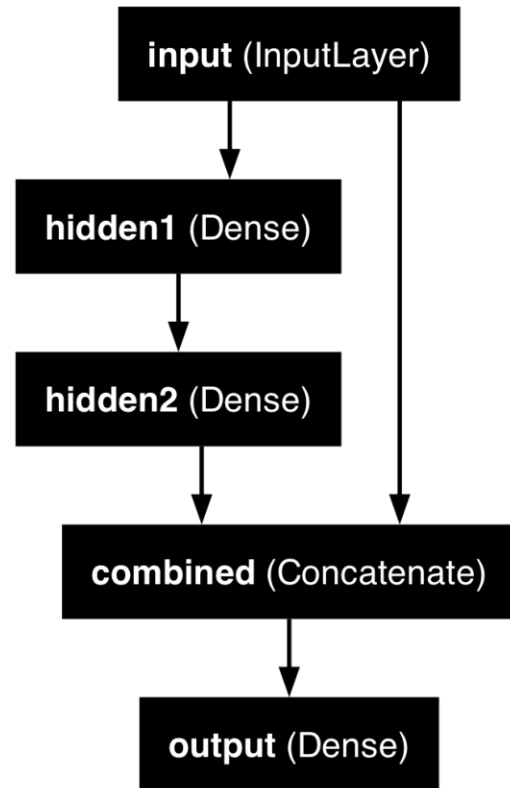
```
1 input_ = Input(shape=X_train.shape[1:], name="input")
2 hidden1 = Dense(30, activation="leaky_relu", name="hidden1")(input_)
3 hidden2 = Dense(30, activation="leaky_relu", name="hidden2")(hidden1)
4 concat = Concatenate(name="combined")([input_, hidden2])
5 output = Dense(1, name="output")(concat)
6 model = Model(inputs=[input_], outputs=[output])
```


Inspecting a complex model

```
1 from keras.utils import plot_model
```

```
1 plot_model(model, show
```

```
1 model.summary(line_length=75)
```



Model: "functional_5"

Layer (type)	Output Shape	Param #	Connected
input (InputLayer)	(None, 39)	0	–
hidden1 (Dense)	(None, 30)	1,200	input[0][0]
hidden2 (Dense)	(None, 30)	930	hidden1[0]
combined (Concatenate)	(None, 69)	0	input[0][0] hidden2[0]
output (Dense)	(None, 1)	70	combined[0]

Total params: 2,200 (8.59 KB)
 Trainable params: 2,200 (8.59 KB)
 Non-trainable params: 0 (0.00 B)

Lecture Outline

- Entity Embedding
- Categorical Variables & Entity Embeddings
- Keras' Functional API
- **French Motor Dataset with Embeddings**
- Scale by Exposure

The desired architecture

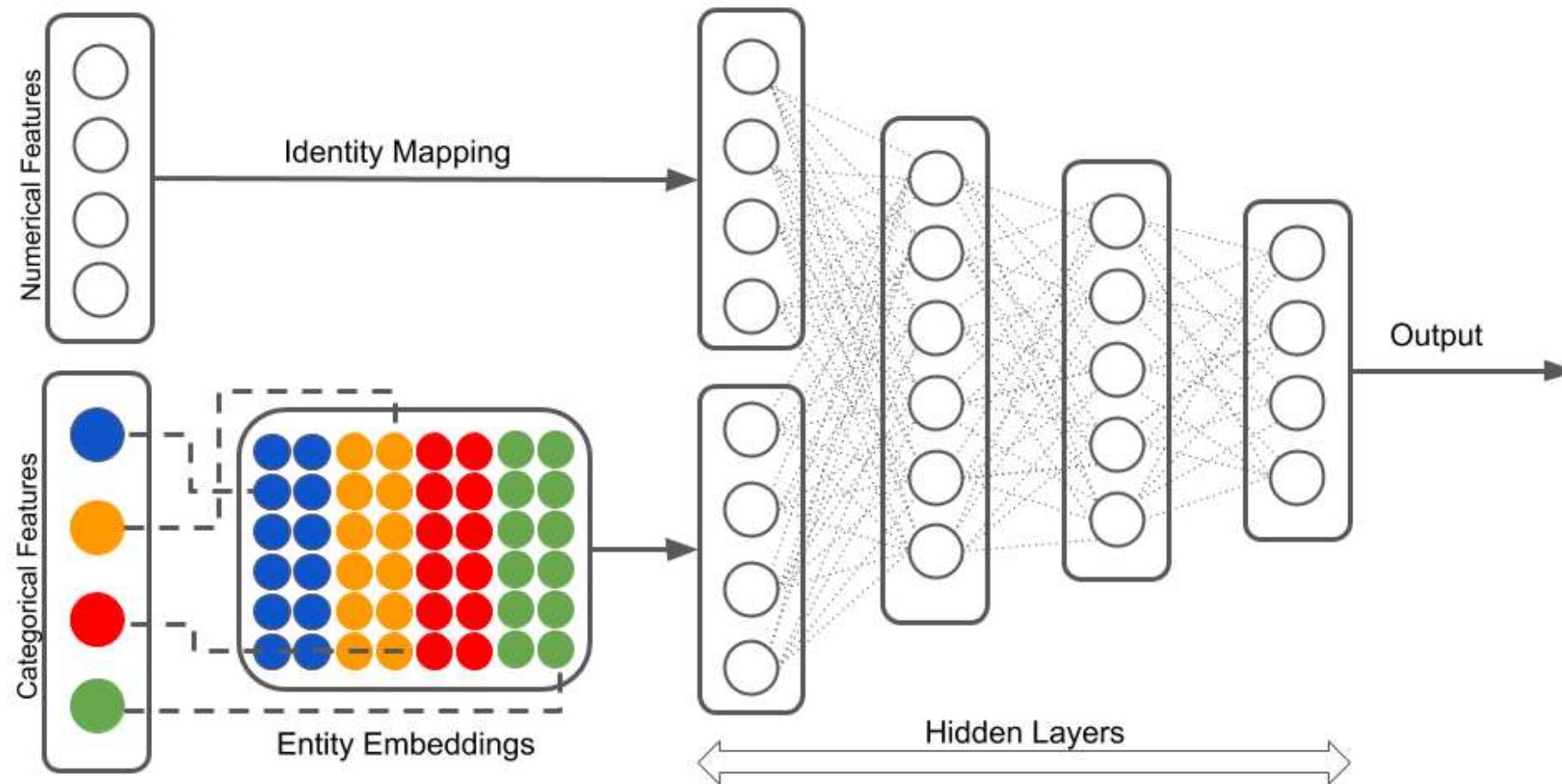


Illustration of a neural network with both continuous and categorical inputs.

Source: LotusLabs Blog, [Accurate insurance claims prediction with Deep Learning](#).

Preprocess all French motor inputs

Transform the categorical variables to integers:

```
1 num_brands, num_regions = X_train_raw[["VehBrand", "Region"]].nunique()
2
3 ct = make_column_transformer(
4     (OrdinalEncoder(), ["VehBrand", "Region", "Area", "VehGas"]),
5     remainder=StandardScaler(),
6     verbose_feature_names_out=False
7 )
8 X_train = ct.fit_transform(X_train_raw)
9 X_test = ct.transform(X_test_raw)
```

Split the brand and region data apart from the rest:

```
1 X_train_brand = X_train["VehBrand"]
2 X_train_region = X_train["Region"]
3 X_train_rest = X_train.drop(["VehBrand", "Region"], axis=1)
4
5 X_test_brand = X_test["VehBrand"]
6 X_test_region = X_test["Region"]
7 X_test_rest = X_test.drop(["VehBrand", "Region"], axis=1)
```

Organise the inputs

Make a Keras `Input` for: vehicle brand, region, & others.

```
1 veh_brand = Input(shape=(1,), name="veh_brand")
2 region = Input(shape=(1,), name="region")
3 other_inputs = Input(shape=X_train_rest.shape[1:], name="other_inputs")
```

Create embeddings and join them with the other inputs.

```
1 from keras.layers import Reshape
2
3 random.seed(1337)
4 veh_brand_ee = Embedding(input_dim=num_brands, output_dim=2,
5     name="veh_brand_ee")(veh_brand)
6 veh_brand_ee = Reshape(target_shape=(2,))(veh_brand_ee)
7
8 region_ee = Embedding(input_dim=num_regions, output_dim=2,
9     name="region_ee")(region)
10 region_ee = Reshape(target_shape=(2,))(region_ee)
11
12 x = Concatenate(name="combined")([veh_brand_ee, region_ee, other_inputs])
```

Complete the model and fit it

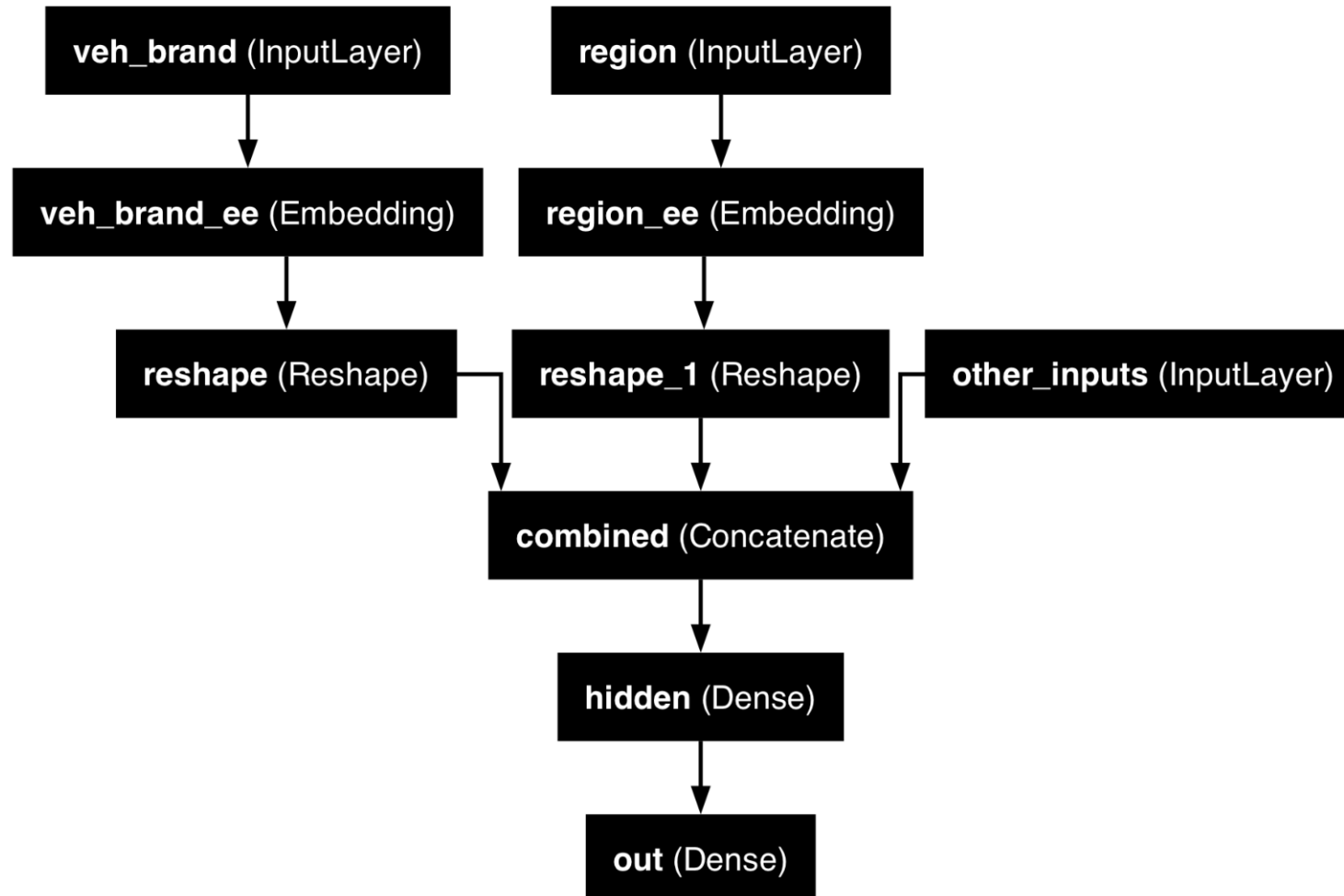
Feed the combined embeddings & continuous inputs to some normal dense layers.

```
1 x = Dense(30, "relu", name="hidden")(x)
2 out = Dense(1, "exponential", name="out")(x)
3
4 model = Model([veh_brand, region, other_inputs], out)
5 model.compile(optimizer="adam", loss="poisson")
6
7 hist = model.fit((X_train_brand, X_train_region, X_train_rest),
8                 y_train, epochs=100, verbose=0,
9                 callbacks=[EarlyStopping(patience=5)], validation_split=0.2)
10 np.min(hist.history["val_loss"])
```

```
np.float64(0.6855398416519165)
```

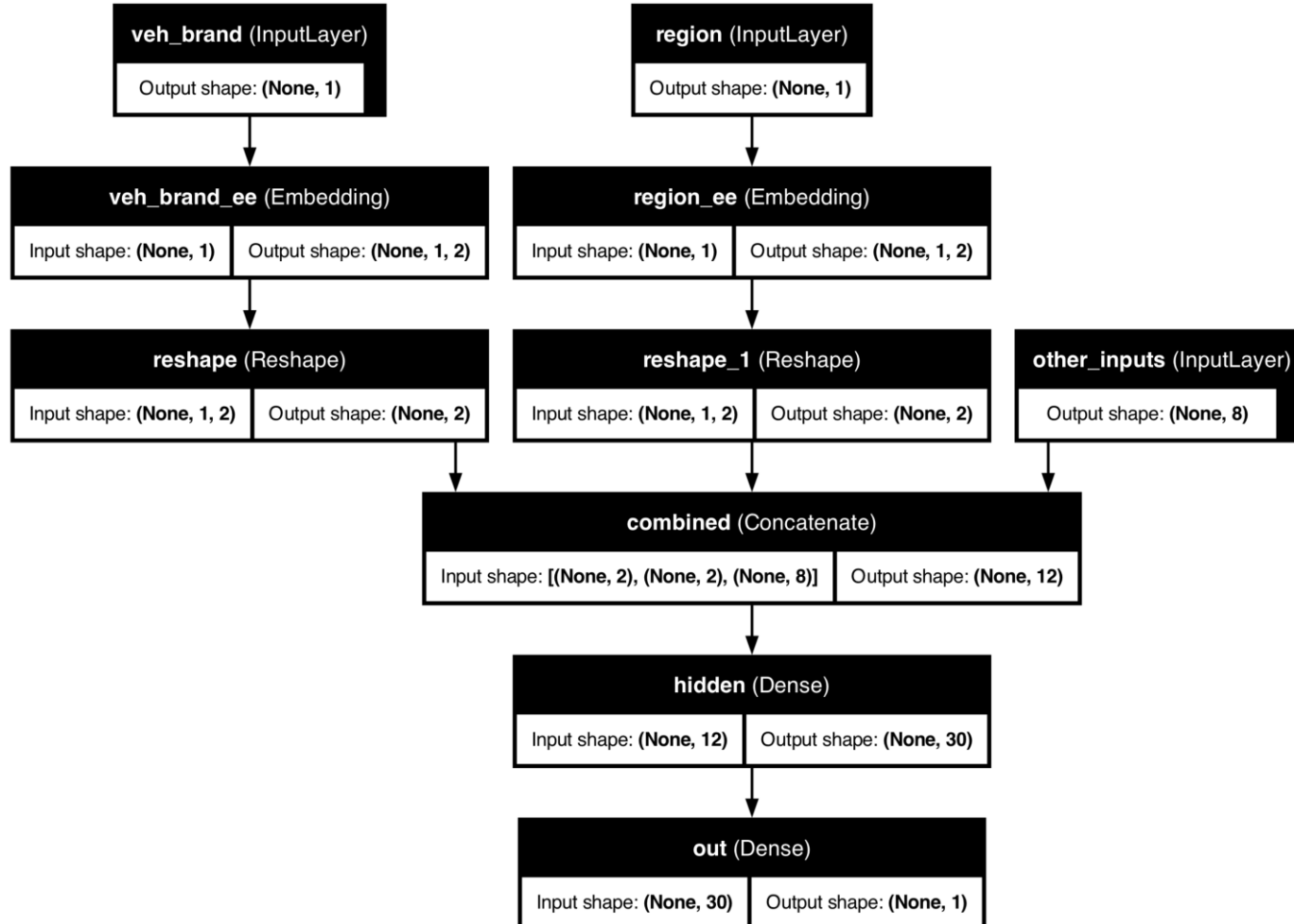
Plotting this model

```
1 plot_model(model, show_layer_names=True)
```



Why we need to reshape

```
1 plot_model(model, show_layer_names=True, show_shapes=True)
```



Lecture Outline

- Entity Embedding
- Categorical Variables & Entity Embeddings
- Keras' Functional API
- French Motor Dataset with Embeddings
- **Scale by Exposure**

Two different models

Have $\{(\mathbf{x}_i, y_i)\}_{i=1, \dots, n}$ for $\mathbf{x}_i \in \mathbb{R}^{47}$ and $y_i \in \mathbb{N}_0$.

Model 1: Say $Y_i \sim \text{Poisson}(\lambda(\mathbf{x}_i))$.

But, the exposures are different for each policy. $\lambda(\mathbf{x}_i)$ is the expected number of claims for the duration of policy i 's contract.

Model 2: Say $Y_i \sim \text{Poisson}(\text{Exposure}_i \times \lambda(\mathbf{x}_i))$.

Now, $\text{Exposure}_i \notin \mathbf{x}_i$, and $\lambda(\mathbf{x}_i)$ is the rate *per year*.

Just take continuous variables

```

1 ct = make_column_transformer(
2     ("passthrough", ["Exposure"]),
3     ("drop", ["VehBrand", "Region", "Area", "VehGas"]),
4     remainder=StandardScaler(),
5     verbose_feature_names_out=False
6 )
7 X_train = ct.fit_transform(X_train_raw)
8 X_test = ct.transform(X_test_raw)

```

Split exposure apart from the rest:

```

1 X_train_exp = X_train["Exposure"]
2 X_test_exp = X_test["Exposure"]
3 X_train_rest = X_train.drop("Exposure", axis=1)
4 X_test_rest = X_test.drop("Exposure", axis=1)

```

Organise the inputs:

```

1 exposure = Input(shape=(1,), name="exposure")
2 other_inputs = Input(shape=X_train_rest.shape[1:], name="other_inputs")

```

Make & fit the model

Feed the continuous inputs to some normal dense layers.

```

1 random.seed(1337)
2 x = Dense(30, "relu", name="hidden1")(other_inputs)
3 x = Dense(30, "relu", name="hidden2")(x)
4 lambda_ = Dense(1, "exponential", name="lambda")(x)

1 out = lambda_ * exposure # In past, need keras.layers.Multiply()[lambda_, exposure]
2 model = Model([exposure, other_inputs], out)
3 model.compile(optimizer="adam", loss="poisson")
4
5 es = EarlyStopping(patience=10, restore_best_weights=True, verbose=1)
6 hist = model.fit((X_train_exp, X_train_rest),
7     y_train, epochs=100, verbose=0,
8     callbacks=[es], validation_split=0.2)
9 np.min(hist.history["val_loss"])

```

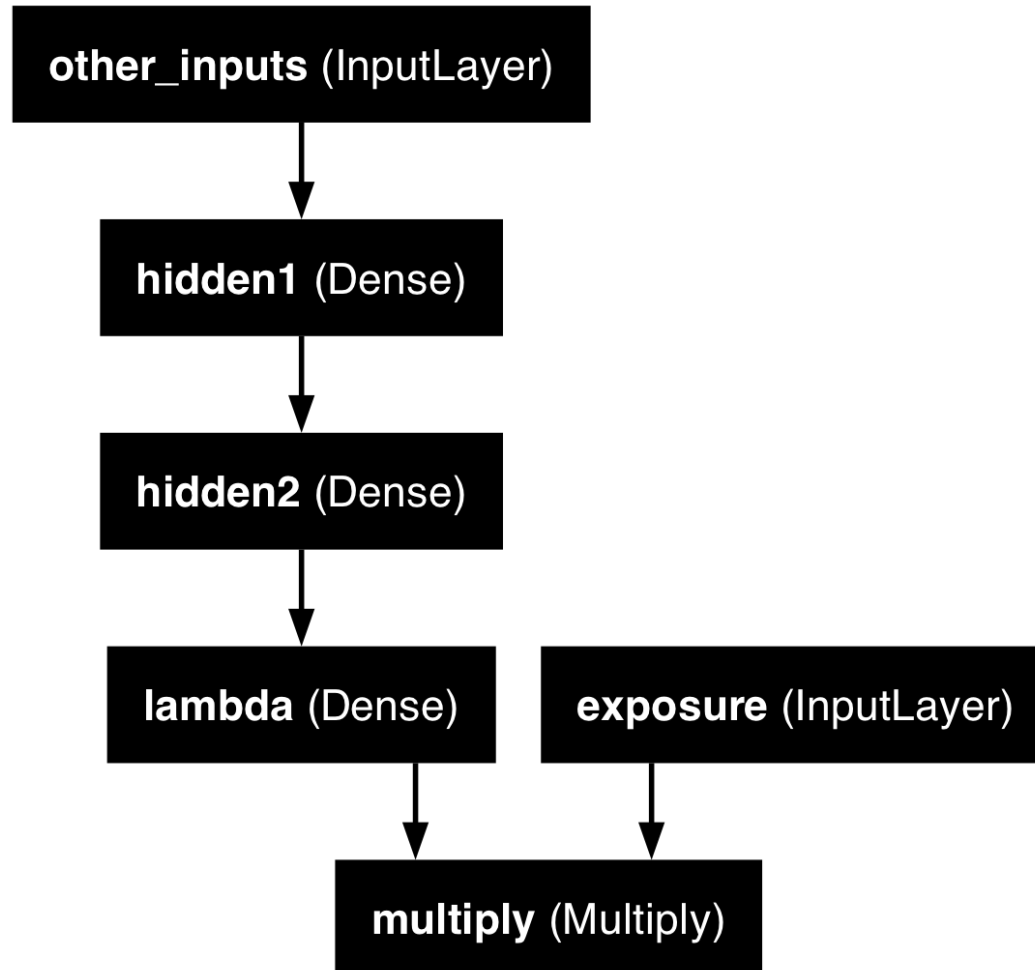
Epoch 29: early stopping

Restoring model weights from the end of the best epoch: 19.

np.float64(0.9100556373596191)

Plot the model

```
1 plot_model(model, show_layer_names=True)
```



Package Versions

```
1 from watermark import watermark
2 print(watermark(python=True, packages="keras,matplotlib,numpy,pandas,seaborn,scipy,torch")
```

Python implementation: CPython

Python version : 3.14.5

IPython version : 9.13.0

keras : 3.14.1

matplotlib: 3.10.9

numpy : 2.4.4

pandas : 3.0.2

seaborn : 0.13.2

scipy : 1.17.1

torch : 2.11.0

Glossary

- entity embeddings
- Input layer
- Keras functional API
- Reshape layer
- skip connection
- wide & deep network